pfi

Python for Fixed Income

COURSE 5C · RISK MODELS

Case Study: Hazard Models

WEEK 1 · DAY 5

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: each firm's own numbers

The Cox model

How much each covariate multiplies the hazard

The discrete-time hazard

A logistic regression on firm-years

The case study

Fitted on earlier study years, would a model have ranked and sized later years' defaults?

18,503 firm-years of 3,504 invented firms, 345 defaults. The generator's formula is known, so the fits can be checked.

Names for a hazard model

A hazard ratio, a fitted hazard, or the model's default probability for that firm-year.

Allowed names

fitted hazard

a synthetic firm's default chance

survival in study years

Never called

a rating view

a real issuer's default chance

a forecast of defaults

Never a credit decision, and never a statement about any real firm.

How a firm-year table works

The problem	Covariates that change every year need one row per firm per year at risk
Each row	The covariates at the start of the year; the event 1 only in the year of default
A clean history	No year twice, no gap, nothing after a default or an exit
Added	event, log_coverage, and early_years (years 2 to 4: the planted age bump)
What comes out	The table every model today is fitted on (person_period)

How the Cox model works

The model	Hazard = a free baseline hazard x exp(b1 x1 + b2 x2 + ...)
The baseline	Any shape at all; only the b's are estimated
Hazard ratio	exp(b): a one-unit rise multiplies the hazard by it, at every time
Firm-years	Each row an interval from its entry time: covariates change year by year
What comes out	A hazard ratio per covariate, with its interval (statsmodels PHReg)

Hazard ratios, and every setting

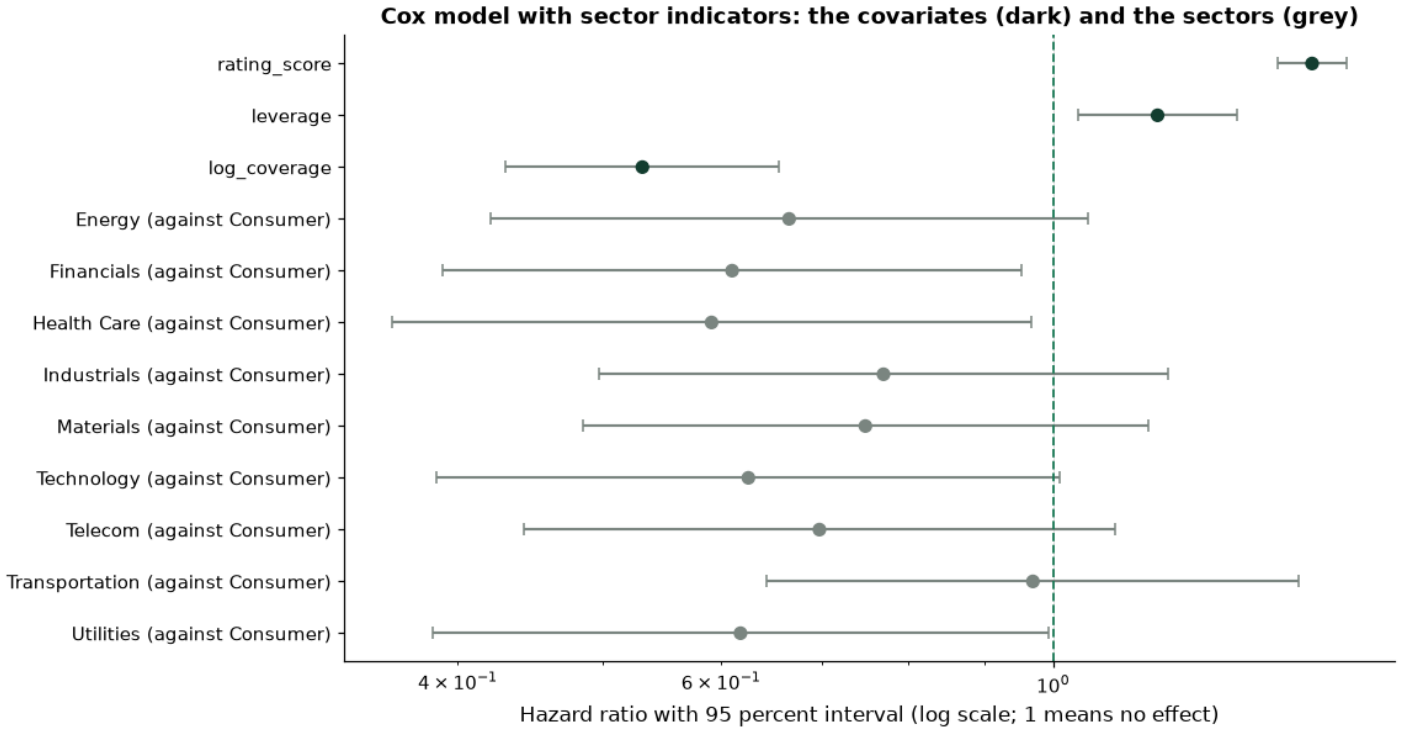
Covariate	Hazard ratio	95% interval
rating_score	1.489	1.411 to 1.571
leverage	1.173	1.039 to 1.324
log_coverage	0.531	0.430 to 0.655

Per notch down the scale, per turn of leverage, and per unit of log coverage: halving coverage multiplies the hazard by more than two.

Settings	Score	Leverage	Log cov.
breslow	0.3914	0.1580	-0.6135
efron	0.3979	0.1592	-0.6334
efron + sector strata	0.3999	0.1622	-0.6446

Whole years tie many defaults, so the method could matter; at a hazard of about 2 percent it barely does.

Sector: an effect that isn't there

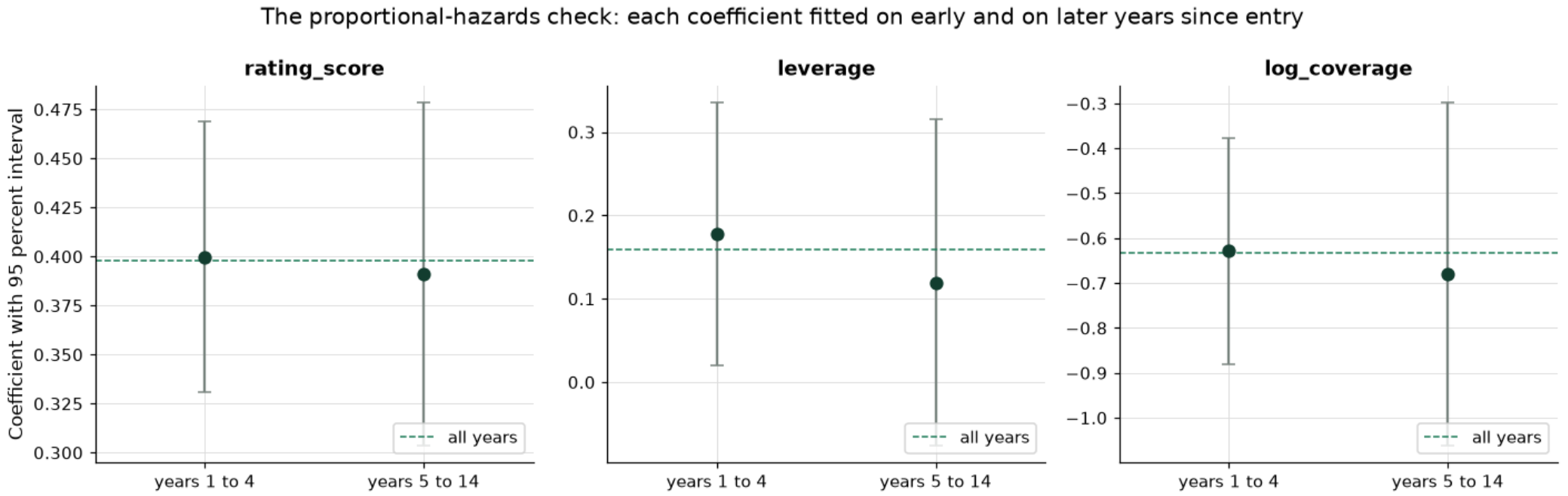


3 of 9

sectors with p below 0.05

The generator has no sector term. Every ratio is measured against Consumer, whose firms happened to default more often: chance in the base, and nine tests at once.

The proportional-hazards check

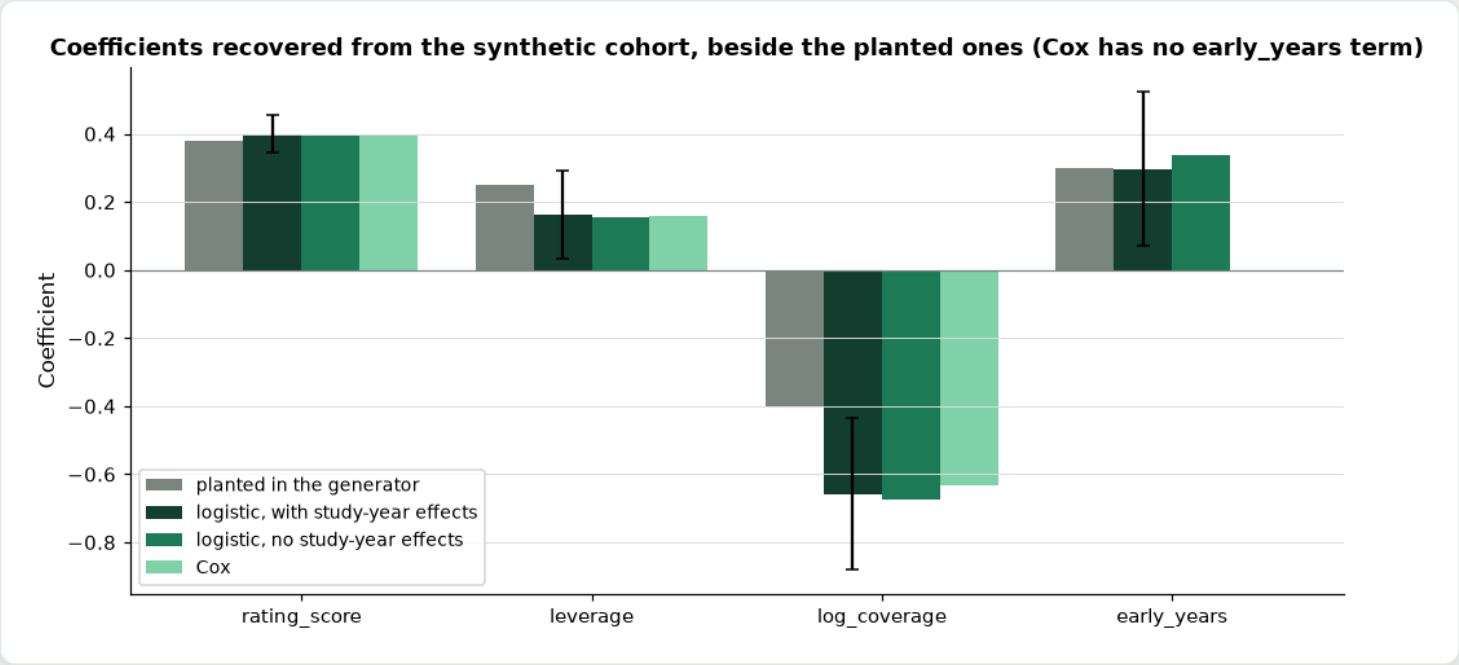


The pairs overlap: one hazard ratio per covariate describes every year. The generator's age bump is in the baseline.

How the discrete-time hazard works

The hazard	The chance a firm at risk at the start of a year defaults during it
The model	A logistic regression on firm-years (Shumway, Journal of Business, 2001)
Each row	Covariates at the start of the year; outcome 1 in the year of default
Unlike Cox	A full default probability per firm-year, chained into a survival curve
Year effects	Study-year indicators soak up what is common to all firms in a year

The fits against the planted truth

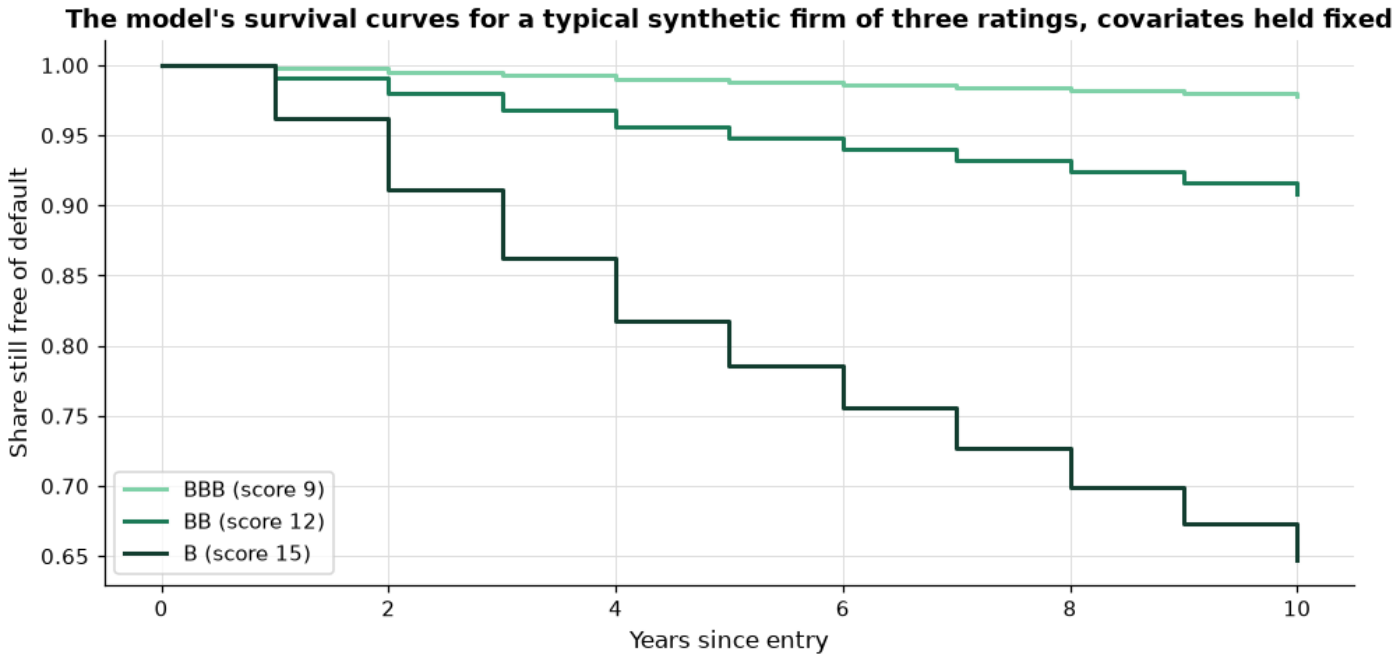


Covariate	Planted	Fitted	Gap
rating_score	0.38	0.400	0.71
leverage	0.25	0.163	-1.31
log_coverage	-0.40	-0.659	-2.28
early_years	0.30	0.297	-0.02

Within two standard errors but one: log coverage, a sampling gap that DATA.md records too.

Here the truth is known. On real data the same table has no planted column.

A typical firm's survival curve



97.9%

typical BBB, past 10 years

64.7%

typical B, past 10 years

hazard_to_survival: day 4's running product, fed with a model's hazards. Covariates held where they started: a simplification.

The case study: rank and size

1. Fit

On study years 1 to k - 1, no year effects

2. Rank

AUC on year k, for k from 8 to 14

3. Size

The model's count, and calibration by decile

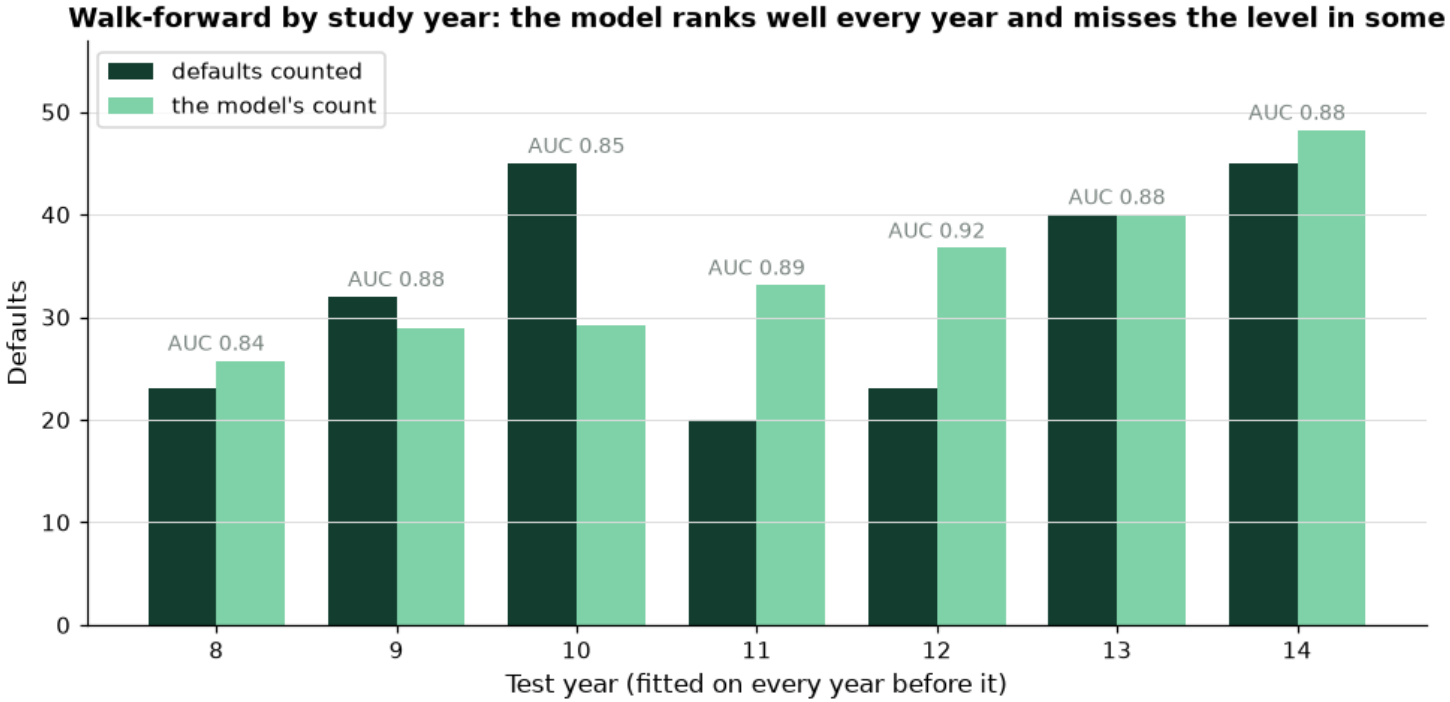
4. Close

What it shows, and where it fails

What the data leave open

The generator's common factor of each study year is in no column: no model on these columns can know whether a year will be a bad one.

Ranking holds, size doesn't



0.877

mean AUC, years 8 to 14

0.60

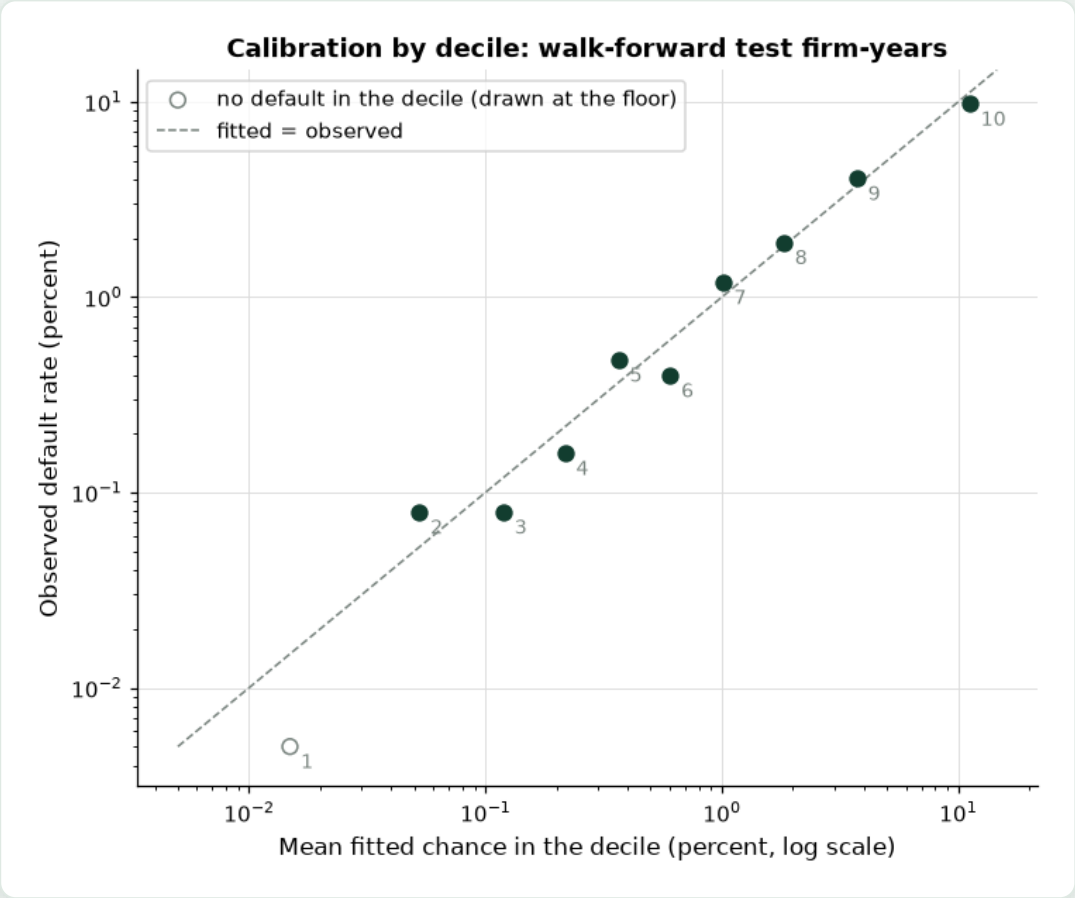
low ratio

1.54

high ratio

Ratio: defaults counted over the model's count. The model knows which firms are weaker, not which years are worse.

Calibration by decile



Decile	Defaults	Fitted %	Observed %
1	0	0.01	0.00
5	6	0.37	0.48
8	24	1.83	1.90
9	51	3.74	4.04
10	123	11.21	9.75

The top decile holds 53.9% of the defaults. The bottom deciles rest on almost no defaults: the low-default problem again.

What the model shows, and where it fails

Shows

Ranks every test year well; fitted chances follow the deciles; coefficients near the planted ones

Fails: the year

The common factor is in no column, so bad and good years are missed both ways

Fails: coverage

Log coverage two standard errors from its planted value: invisible to one fit

Fails: sector, low end

Sector effects that are not there; small chances resting on the line, not on defaults

Past study years of invented firms, described. The held-out years open in the capstone.

The model nobody noticed for a while

D. R. Cox, Journal of the Royal Statistical Society, Series B, 1972

"Regression Models and Life-Tables"

In Cox's own words to Nancy Reid, Statistical Science, 1994

"Nobody took much notice of the 1972 paper for a while, until various people started to write software which was widely useful." Its first benefit, he said: "Handling in-study covariates, that is, time-dependent covariates".

Source: N. Reid, A Conversation with Sir David Cox, Statistical Science 9 (3), 1994, pages 449 and 450.

Excel side by side

Excel side by side (tested in Excel)

`=1/(1+EXP(-SUMPRODUCT(B1:B5,C1:C5)))`	fitted hazard 0.005706
`=EXP(E1)`	hazard ratio 1.4887
`=EXP(E1-NORM.S.INV(0.975)*E2)`	95% low 1.4111
`=PRODUCT(1-G1:G10)`	survival 0.9082
`=AVERAGEIFS(K2:K12615,J2:J12615,10)`	top decile 9.75%

statsmodels uses NORM.S.INV(0.975) exactly; Excel has no Cox model and no AUC function.

Save the day: tag the end of week 1

1. Commit

mlkit.py, riskkit.py, and its tests

2. Tag

A name for one commit that stays put

3. List

git tag -n and git log --decorate

```
git tag -a week1 -m "Course 5C week 1: simulation and default"
```

week1 will always mean the module as it stood at the end of the week.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

What is drawn, from which inputs, in which unit: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: an input from the firm's future: 1552 firms' year-10 inputs change when later years are added.

Code that runs is not the same as code that is right.

What you can do now

- Build and check a firm-year table (person_period)
- Fit Cox with covariates that change; read hazard ratios
- Check proportional hazards; fit a discrete-time hazard
- Chain hazards into a curve (hazard_to_survival)
- Score walk-forward by AUC and calibration; 24 tests pass

Week 2, Day 1 Outliers: robust rules